PHASE 1

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

PHASE 2

PHASE 3

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/17/6b615f416127a5e3963081e1ca30821c35c0e139fea65b85200eafa7f91435
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/81/17905101dcdcedf57b4ae44bf1605cebb4bccee0c366a683dddb72441b8612
.pes/objects/8a/1519e4ecfc91ecc355a0b9f4f8d3a9f5078fff0eb502b4ed5643d64f82f7b4
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ cat .pes/refs/heads/main
b7a5373e3d40d0bafe6f1ae65761ccf578d8bc7f6cd84e50f85ef7d23fad2c6f
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/9115fd437e65500842695fffd4904a53f815adec5d23c548c4eb1bca9b3d20
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/26/b5efc99f9c8a5057cdfdccaa5b2aa5f4b7f38fb95c3c6f5f30494bad5eea9f
.pes/objects/5c/77286fb4268c6dfd500b802075e6cc92f5a0c6daa61b29fd116ecea3c4a0c7
.pes/objects/65/d137c1db2e0aa15bae43421c70a2ec9f31507a740bfb7232863111c0972707
.pes/objects/9b/15b8eaef7d1d839356cc58644dc9a80b92866f7f0fd53d1363b63d30756ab8
.pes/objects/a5/4581c02c0569fa870f342161a9bba229f62679e28bcfddefc38df2ff3f3703
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12
.pes/objects/b7/a5373e3d40d0bafe6f1ae65761ccf578d8bc7f6cd84e50f85ef7d23fad2c6f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./pes log
commit b7a5373e3d40d0bafe6f1ae65761ccf578d8bc7f6cd84e50f85ef7d23fad2c6f
Author: PES User <pes@localhost>
Date:   1776682225

    Add farewell

commit 26b5efc99f9c8a5057cdfdccaa5b2aa5f4b7f38fb95c3c6f5f30494bad5eea9f
Author: PES User <pes@localhost>
Date:   1776682180

    Add world

commit a54581c02c0569fa870f342161a9bba229f62679e28bcfddefc38df2ff3f3703
Author: PES User <pes@localhost>
Date:   1776682073

    Initial commit
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ cat .pes/index
100644 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776681773 12 file1.txt
100644 5c77286fb4268c6dfd500b802075e6cc92f5a0c6daa61b29fd116ecea3c4a0c7 1776681870 8 file2.txt
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ echo "hello world" > file1.txt
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./pes add file1.txt
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ echo "foo bar" > file2.txt
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./pes add file2.txt
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./pes status
Staged changes:
    staged: file1.txt
    staged: file2.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: tree.h
    untracked: test_objects.c
    untracked: index.c
    untracked: pes.c
    untracked: test_tree
    untracked: test_sequence.sh
    untracked: README.md
    untracked: tree.c
    untracked: test_objects
    untracked: .gitignore
    untracked: index.h
    untracked: commit.h
    untracked: test_tree.c
    untracked: commit.c
    untracked: Makefile
    untracked: pes.h
    untracked: object.c
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

```
yashas-s@yashas-s-VirtualBox:~/PES1UG24CS545-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

PHASE 4

PHASE 5 AND PHASE 6 [QnA]

Phase 5: Branching and Checkout

Q5.1 — How does `pes checkout <branch>` work?

When switching branches, the following changes occur:

- The `.pes/HEAD` file is updated to point to the new branch
- The working directory is updated to match the target commit
- The index is reloaded to reflect the new tree

Steps involved:

1. Read the current HEAD to determine the current branch and commit
2. Identify the target branch or commit
3. Load the target commit's tree object
4. Update files in the working directory
5. Update `.pes/HEAD` to the new branch
6. Reload the index to match the new tree

Complexities:

- Handling modified (dirty) files
- Recursive directory updates
- Maintaining consistency if interrupted

Q5.2 — How are dirty file conflicts detected?

A file is considered conflicting if:

1. It exists in the index
2. Its current content differs from the stored hash
3. The target branch contains a different version

Detection process:

- Compute the hash of the file in the working directory
- Compare it with the index hash
- Compare it with the target branch hash

If all differ, a conflict is detected and checkout is prevented.

Q5.3 — What is Detached HEAD and how can it be recovered?

Detached HEAD occurs when HEAD points directly to a commit instead of a branch.

In this state:

- New commits are not associated with any branch
- They can become unreachable if not saved

Recovery methods:

Before leaving detached state:
pes branch recover-work

After leaving:

- Search `.pes/objects` for commit objects
- Identify dangling commits
- Create a new branch pointing to the required commit

Phase 6: Garbage Collection and Space Reclamation

Q6.1 — How to find and delete unreachable objects?

Garbage collection uses the **Mark and Sweep algorithm**.

Mark Phase:

- Start from all branch references
- Traverse commits, trees, and blobs
- Store all reachable objects

Sweep Phase:

- Traverse all objects in `.pes/objects/`
- Delete objects not present in the reachable set

Data Structure:

- A hash set is used for fast lookup of object IDs

Estimated scale:

- For large repositories, around 400,000 to 700,000 objects may be processed

---

Q6.2 — What is a race condition between GC and commit?

A race condition occurs when garbage collection runs while a commit is being created:

1. Commit writes a new object
2. GC runs before the commit completes
3. GC deletes the new object (thinking it is unused)
4. Commit references a deleted object

This leads to repository corruption.

How Git avoids this:

- Recently created objects are not deleted
- Lock files are used during operations
- Reachability is rechecked before deletion

Simplified solution:

- Avoid running GC during active writes
- Detect temporary or lock files before deletion

Conclusion

Phases 5 and 6 focus on advanced version control concepts such as branch management, conflict detection, and storage optimization. These mechanisms ensure data integrity, efficient storage, and reliable version tracking in a version control system.